

AI Lab # 4

This lab extends depth-first search by implementing Depth-Limited Search and Iterative Deepening Search to efficiently explore state spaces with controlled depth.

Artificial Intelligence (LAB)

Instructor: Mubbashir Ahmed

Email: mubbashir.ahmed@iqra.edu.pk

Date: 08 August, 2026

Today's Agenda

01

What is DLS?

02

Algorithm Steps and Example

03

What is IDS?

04

Algorithm Steps and Example

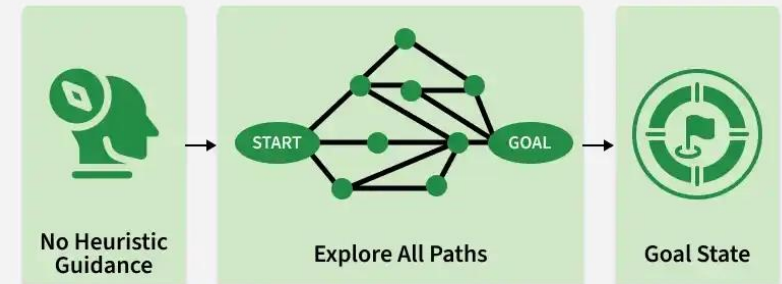
05

Lab Tasks

Uninformed Search

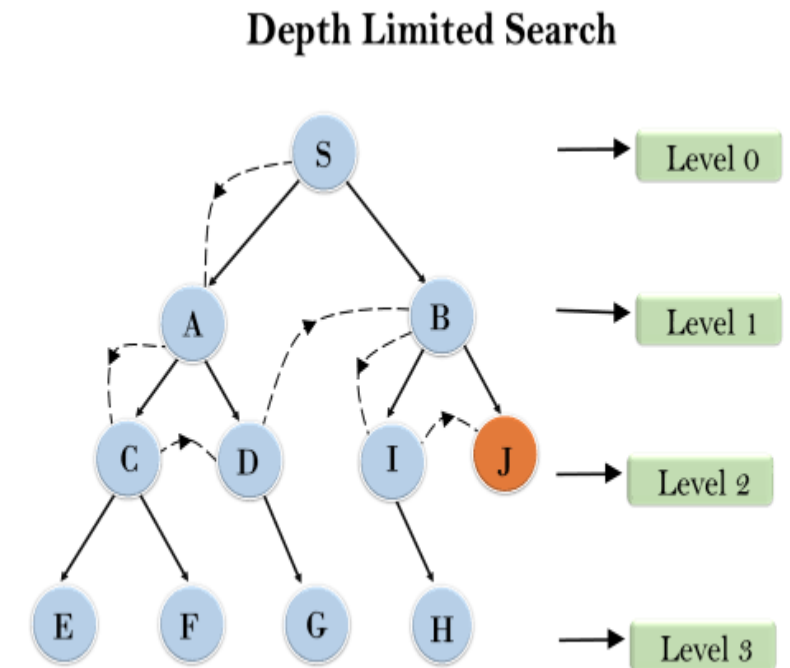
- Uninformed search algorithms also called blind search algorithms.
- They have no additional information about goal location except the problem definition.
- They explore the search space blindly.
- Examples are BFS, DFS, UCS

Uninformed Search in Artificial Intelligence



What is DLS?

- DLS is an uninformed search algorithm.
- It works like DFS but stops exploring once a set depth limit is reached.
- Prevents infinite loops in deep or cyclic graphs by cutting off the search early.



Algorithm Steps

1

Push the start node: Put starting node onto the stack with depth 0

2

Pop and visit: Take the front item from the stack and add it into the visited list

3

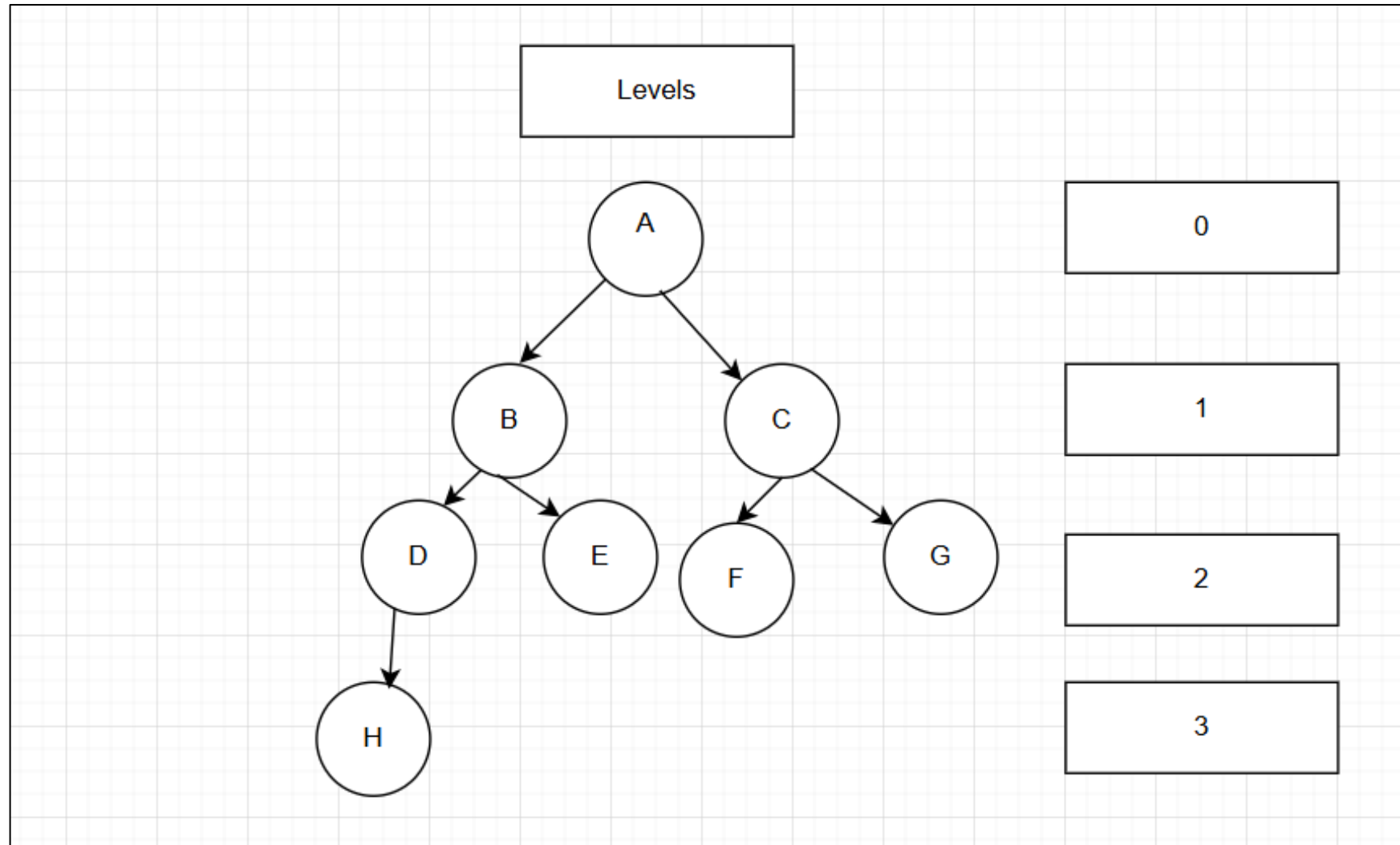
Check goal found or depth limit: If the goal is found or the current depth equals the limit, stop traversing the graph

4

Push unvisited neighbors: Find adjacent nodes that are not visited and add them onto the stack with current depth + 1

5

Repeat until the goal is found or stack is empty: Keep repeating step 2 to 4 until the goal is found or the stack is empty



Example of DLS

Now that we've covered DLS, let's move towards practical code-based example

DLS

Depth Limiting Search on nodes A, B, C, D, E, F, G, H. Starting node is A and Goal node is G.

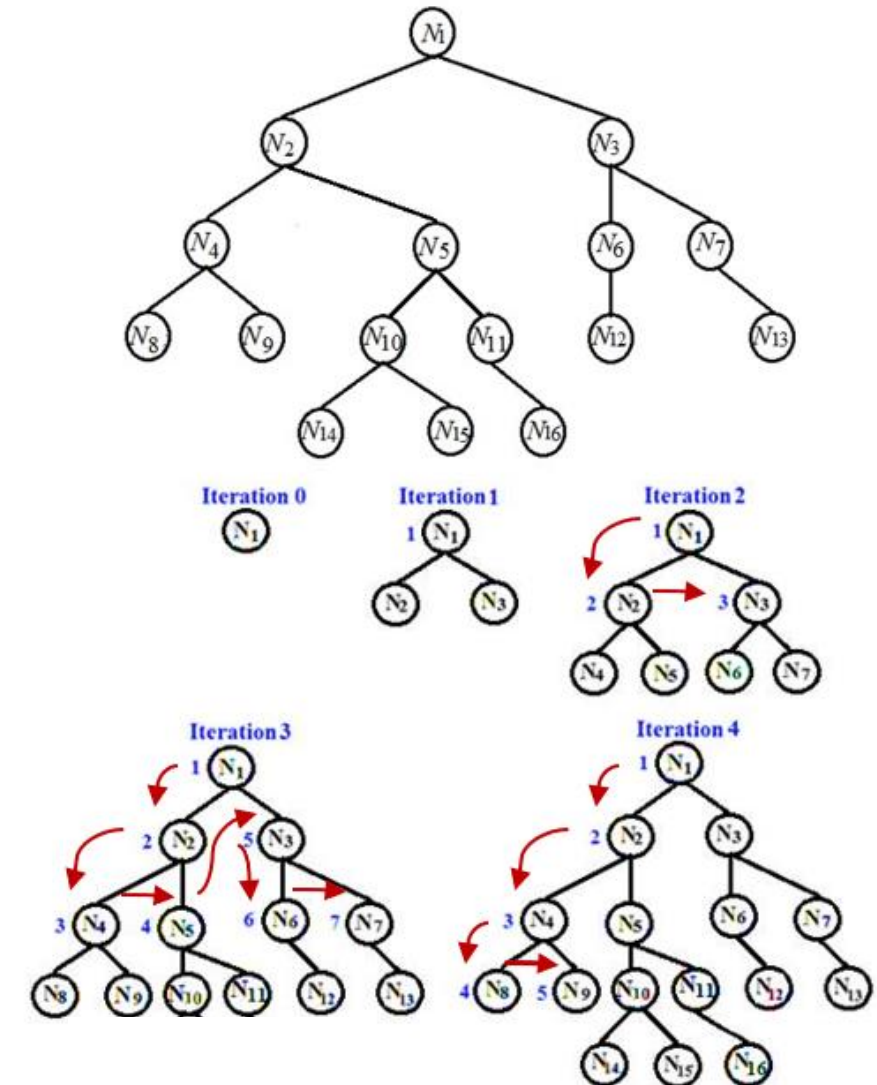


```
graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': ['H'], 'E': [], 'F': [], 'G': [], 'H': [] }  
def depth_limited_search(start, goal, limit):  
    visited = []  
    stack = [(start, 0)]  
  
    while stack:  
        node, depth = stack.pop()  
        if node not in visited:  
            visited.append(node)  
            if node == goal:  
                return visited  
  
            if depth < limit:  
                for child in reversed(graph[node]):  
                    stack.append((child, depth + 1))  
    return visited
```

```
visited_nodes = depth_limited_search('A', 'G', 2)  
print("Visited Nodes:", visited_nodes)
```


What is IDS?

- IDS is an uninformed search algorithm.
- It works by running DLS repeatedly, increasing the depth limit by 1 each time until the goal is found.
- Combines the memory efficiency of DFS and DLS with the ability to search level by level just like BFS.



Algorithm Steps

1

Push the start node: Put starting node onto the stack with depth 0

2

Run DLS with current limit: Perform Depth-Limited Search from the start node using the current depth limit (this reuses your DLS steps 1–5 internally)

3

Check goal found: If the goal is found during this DLS run, stop and return the result

4

Increase the limit: If the goal is not found, increase the depth limit by 1

5

Repeat until goal is found or limit reached: Keep repeating steps 2 to 4 with the increasing limit until the goal is found

Example of IDS

Now that we've covered IDS, let's move towards practical code-based example

IDS

Iterative Deepening Search code utilizes our Depth Limited Search function.



```
def iterative_deepening_search(start, goal, max_limit):  
    for limit in range(max_limit + 1):  
        visited, goal_found = depth_limited_search(start, goal, limit)  
  
        if goal_found:  
            return visited, limit  
    return visited, None  
  
visited_nodes, found_at_limit = iterative_deepening_search('A', 'G', 5)  
print("Final Visited Order:", visited_nodes)
```

Lab Task: Depth Limited Search and Iterative Deepening Search (Uninformed Search)

- *Write a DLS program to find goal 'G' in the given graph using a depth limit of 2, and display the visited nodes.*
- *Write an IDS program to find goal 'G' in the given graph by repeatedly applying DLS with increasing depth limits, and display the depth at which the goal 'G' is found.*

Given Graph:

$A \rightarrow B, C$

$B \rightarrow D, E$

$C \rightarrow F, G$

$D \rightarrow H$

$E \rightarrow G$

$F \rightarrow G$

Thank You

Questions or discussions are welcomed!